Introduction to Pseudo-Random Number Generator (PRNG)

Introduction

- A Pseudo-Random Number Generator (PRNG) is an algorithm that
  - generates a sequence of numbers approximating the properties of random numbers.
- Unlike true random number generators,
  - PRNGs are deterministic and rely on an initial value called a seed.
- Random numbers are generated using developed algorithms
  - that's why they are called Pseudo Random Numbers

- **Key Characteristics:**
  - Deterministic (generated using algorithms)
  - Reproducible (same seed → same output)
  - Faster than True Random Number Generators (TRNGs)

Common Uses of PRNGs

- **Cryptography**: Generating encryption keys; Creating session keys in secure communication (SSL/TLS); Generating nonces and random challenges (like in Zero-Knowledge Proofs); Password salts
- **Authentication Protocols:** Random challenge values in protocols; OTP generation; Digital signature schemes
- **Simulations & Modeling**: Weather simulations; Scientific experiments; Network simulations; Monte Carlo methods
- **Gaming**: Randomized game mechanics; procedural content generation
- **AI & Machine Learning**: Random initialization in neural networks
- **Testing & Statistical Sampling**: Random test case generation

True Random Numbers (Vs PRN)

- Generated from physical processes (e.g., atmospheric noise, radioactive decay, thermal noise)
- Unpredictable & non-reproducible
- Requires special hardware (e.g., quantum random number generators, hardware TRNGs)
- Used in high-security applications (e.g., cryptographic key generation, secure communications)

Security Aspects of PRNGs

- **Vulnerabilities of PRNGs**
  - 🔹 **Predictability**: If the seed is known, the entire sequence can be reconstructed.
  - 🔹 **Low Entropy**: Weak sources of randomness make them vulnerable to attacks.
  - 🔹 **Seed Attacks**: If an attacker determines the seed, they can predict all future values.
- **Cryptographically Secure PRNGs (CSPRNGs)**
  - ✅ Use cryptographic algorithms to enhance randomness and security
  - ✅ Must pass stringent randomness tests (e.g., NIST tests)
  - ✅ Examples: Blum-Blum-Shub (BBS), Yarrow, Fortuna

Congruence Methods in PRNGs

- One of the oldest and simplest PRNG algorithms:

$$\boldsymbol{x_{n+1} = (a x_n + c) \, mod \, m}$$

where:

  - x_n is previous number
  - a is multiplier
  - c increment
  - m is modulus

- Fast, easy to implement
- Poor randomness properties, predictable if parameters are known

Example

$$x_{n+1} = (ax_n + c)\ mod\ m$$

Given Parameters:

- Seed $(x_0) = 7$
- Multiplier $(a) = 5$
- Increment $(c) = 3$
- Modulus $(m) = 16$

Compute 10 sequence of random number using C

Results of first 10 sequences

We are given the linear congruential generator (LCG):

$$x_{n+1} = (a \cdot x_n + c) \mod m$$

with:

$$x_0 = 7, a = 5, c = 3, m = 16$$

We want $x_1, x_2, \ldots, x_{10}$.

Step 1 – x_1

$$x_1 = (a \cdot x_0 + c) \mod m$$
$$x_1 = (5 \times 7 + 3) \mod 16$$
$$= (35 + 3) \mod 16$$
$$= 38 \mod 16$$

$16 \times 2 = 32$, remainder $38 - 32 = 6$

$$x_1 = 6$$

Step 2 – x_2

$$x_2 = (a \cdot x_1 + c) \mod m$$
$$x_2 = (5 \times 6 + 3) \mod 16$$
$$= (30 + 3) \mod 16$$
$$= 33 \mod 16$$

$16 \times 2 = 32$, remainder $33 - 32 = 1$

$$x_2 = 1$$

Step 3 – x_3

$$x_3 = (a \cdot x_2 + c) \mod m$$
$$x_3 = (5 \times 1 + 3) \mod 16$$
$$= (5 + 3) \mod 16$$
$$= 8 \mod 16$$
$$x_3 = 8$$

Step 4 – x_4

$x_4 = (a \cdot x_3 + c) \bmod m$

$x_4 = (5 \times 8 + 3) \bmod 16$

$= (40 + 3) \bmod 16$

$= 43 \bmod 16$

$16 \times 2 = 32$, remainder $43 - 32 = 11$

$x_4 = 11$

Step 5 – x_5

$x_5 = (a \cdot x_4 + c) \bmod m$

$x_5 = (5 \times 11 + 3) \bmod 16$

$= (55 + 3) \bmod 16$

$= 58 \bmod 16$

$16 \times 3 = 48$, remainder $58 - 48 = 10$

$x_5 = 10$

Step 6 – x_6

$x_6 = (a \cdot x_5 + c) \bmod m$

$x_6 = (5 \times 10 + 3) \bmod 16$

$= (50 + 3) \bmod 16$

$= 53 \bmod 16$

$16 \times 3 = 48$, remainder $53 - 48 = 5$

$x_6 = 5$

Step 7 – x_7

$x_7 = (a \cdot x_6 + c) \bmod m$

$x_7 = (5 \times 5 + 3) \bmod 16$

$= (25 + 3) \bmod 16$

$= 28 \bmod 16$

$16 \times 1 = 16$, remainder $28 - 16 = 12$

$x_7 = 12$

Step 8 – x_8

$x_8 = (a \cdot x_7 + c) \bmod m$

$x_8 = (5 \times 12 + 3) \bmod 16$

$= (60 + 3) \bmod 16$

$= 63 \bmod 16$

$16 \times 3 = 48$, remainder $63 - 48 = 15$

$x_8 = 15$

Step 9 – x_9

$x_9 = (a \cdot x_8 + c) \bmod m$

$x_9 = (5 \times 15 + 3) \bmod 16$

$= (75 + 3) \bmod 16$

$= 78 \bmod 16$

$16 \times 4 = 64$, remainder $78 - 64 = 14$

$x_9 = 14$

Step 10 – x_{10}

$x_{10} = (a \cdot x_9 + c) \mod m$

$x_{10} = (5 \times 14 + 3) \mod 16$

$= (70 + 3) \mod 16$

$= 73 \mod 16$

$16 \times 4 = 64$, remainder $73 - 64 = 9$

$x_{10} = 9$

Sequence:

$$x_1 = 6,\ x_2 = 1,\ x_3 = 8,\ x_4 = 11,\ x_5 = 10,\ x_6 = 5,\ x_7 = 12,\ x_8 = 15,\ x_9 = 14,\ x_{10} = 9$$

If we want to output them as random integers in range $[0, m-1]$:

$$\{6, 1, 8, 11, 10, 5, 12, 15, 14, 9\}$$

Python code

```
def lcg(seed, a=1664525, c=1013904223,
m=2**32, n=10):
    numbers = []
    X = seed
    for _ in range(n):
        X = (a * X + c) % m
        numbers.append(X)
    return numbers

# Example run with seed = 42
print(lcg(seed=42))
```

Exercise

Exercise 1
Given:

$$x_{n+1} = (7x_n + 5) \mod 11$$

Seed $x_0 = 3$
Compute the first 6 random numbers

Exercise 2
Given:

$$x_{n+1} = (3x_n + 8) \mod 20$$

Seed $x_0 = 2$
Compute the first 8 random numbers

Exercise 3
Given:

$$x_{n+1} = (4x_n + 1) \mod 9$$

Seed $x_0 = 2$
Compute the sequence until it repeats.